

Robot Sumo Simulation: Test Plan Document

Team Members

Adebare Kofo-Abayomi: akofoaabyomi2022@my.fit.edu

Avlynn Wade: awade2024@my.fit.edu

Faysal Zaki: fzaki2023@my.fit.edu

Joshua Oziel Fernández Alvarado: jfernandezal2023@my.fit.edu

Document Identifier

Florida Institute of Technology

Department of Electrical Engineering and Computer Science

CSE:4101: Software Development Projects 1

Dr. Hanks, thanks@fit.edu

September 28, 2026

Table of Contents

1. Introduction
 - 1.1. Scope
 - 1.2. References
 - 1.3. Context
2. Details
 - 2.1. Reinforcement Learning Tests (RLT)
 - 2.2. Reinforcement Learning Configuration (RLC)
 - 2.3. Physics Engine Tests(PET)

Section 1.0 Introduction

This document lists planned test cases for each functional requirement mentioned in the SRS for Robot Sumo. This test documentation is a part of milestone 1 deliverables, which are focused on prototyping, research, and planning. As such, test cases here are high-level drafts with implementation specifics kept broad, as they are subject to change. More details on the test document's scope, references, and context are included in their respective sections.

Test Case Naming Convention

Test cases will follow this regex as a naming convention L - xxx where L is any 3 capital characters $[A-Z]\{3\}$ and where x is any 3 concatenated integers $[0-9]\{3\}$. L and xxx are separated out by “-” which is meant to be interpreted in its literal form. Test cases are to be read as some requirement label L , and some number xxx . Each label L , is to logically correspond to the broad requirements in the SRS it is testing for, but this isn't a hard standard, especially since integration tests are anticipated for future milestones, so tests can be related to more than one requirement. The last two symbols in xxx are to be numbered as created relative to its label. The first symbol in xxx denotes a subsection of tests within the section that the label.

Example) A test labeled AXE-000 is the first test case written in the context of a video game testing axe mechanics, so requirement testing related to that is labeled AXE.

Further, tests such AXE-000, AXE-001, AXE-002 test very similar but slightly different cases. Another series of tests that cover the same requirement but tests a different area of code with different inputs and outcomes would be labeled AXE-100, AXE-101, AXE-102, and so on.

Section 1.1 Scope

Testing will generally cover implementations and features written and built using tools provided by the physics engine of choice and the reinforcement learning API. That is to say, the testing will strictly cover what is written for the project. The project itself aims to deliver an executable that has a ready-to-run and configurable physics environment that also has interfaces already established which allow RL agents to train either locally or offload training to Panther AI. These features are to be easily accessible from a GUI, and configurations are to be done from a separate file.

Test cases will mainly focus on 2 areas of requirements: RL training functionality and user input functionality. The program must correctly integrate RL training features to allow RL reinforcement learning to take place. The second area of test cases relates to systems that handle user interaction, such as the GUI, and whether those systems are functional.

Section 1.2 References

Functional Requirement References

Reinforcement Learning Tests (RLT)

Section 3.2.2 Single-Agent RL Training

Section 3.2.3 Progressive Training

Section 3.2.6 MARL and Custom Agent Implementation

Reinforcement Learning Configuration (RLC)

Section 3.2.1 Match Configuration

Section 3.2.4 Agent Visualization

Physics Engine Tests(PET)

Section 3.2.4 Agent Visualization

Section 1.3 Context

The planned tests are described at a high level. Test cases prefixed as RLT(Reinforcement Training Tests) relate to testing reinforcement learning features and functionality. Most input parameters listed reference input that should be possible from the GUI before a sumo match. These inputs are not explicitly named, but instead their functions are described. Test cases run here are different configurations of simulated sumo matches. Since results from an RL agent are non-deterministic and training can take a substantial amount of time, test cases usually verify a small test run to see if the RL API and built interfaces are working correctly. Averages of how much each game state differs are also taken to see if the RL agent is at least trying to adapt and isn't immediately stuck, or if there isn't another issue with the API. Game states, when referenced here, refer to general simulation data that gives the exact state of every simulated object at that time. Commonly, video games in these engines measure game state in discrete ticks, with a tick generally referring to a loop or iteration of the game's main loop.

Test cases prefixed as RLC (Reinforcement Learning Configuration) directly aim to test configuration features themselves. Requirements related to GUI and user-initiated configuration are tested. Test cases prefixed as PET relate to engine-facing requirements. As of now, that just includes Agent Visualization.

Section 2.0 Details

Section 2.1 Reinforcement Learning Tests (RLT)

RLT - 00X Single Agent Training Tests

RLT-001 Single Agent Training Test

Input

- Default physics environment configuration
 - Assuming flat ring environment
 - No obstacles
- Mode: Scripted Opponent vs AI agent
 - Default physics body for each
 - Untrained/Fresh RL Agent
 - Default RL training algorithm
 - Opponent scripted to simply move toward opponent
- Simulation ran in headless mode for an arbitrary amount of time on a local machine
 - Likely cannot test agent until results reach a peak due to hardware limitations

Expected Outcomes

- RL agent parameters change after each match and differ from beginning parameters by a defined tolerance
- RL interface does not give default output as if the simulation wasn't running or providing information about gamestate
 - Output isn't a default from non-functioning implementation that doesn't throw
 - Output differs from each other by a tolerance for some amount of time in a row
 - Not expected to reach this due to agent learning reaching a peak due to limited run time. Having very similar output likely indicates learning is somehow hindered or non-functional
 - Could also measure this as an average once simulation ends
 - The idea of keeping this as a tolerance over a sequence is to have the test exit out early when a sequence of similar outputs is detected to save time.
- An average of how different game states were from each other passes some tolerance

- Given this is the default configuration with a very simple scripted AI against the RL agent, if the gamestates between each match did not differ a substantial amount, then it'll be obvious that the RL agent was not functioning

RLT-10X Progressive Training

RLT-100 RL Agent vs scripted opponent

Input

- Identical to input specified in RLT-000
- Differences being tests will need to be run for longer. Time could vary depending on implementation and if local hardware is used.

Expected Outcomes

- The tracked historical wins and losses from the RL agent's perspective should gradually increase until reaching a peak

RLT-101 RL Agent vs previously trained version of itself

Input

- Refer to RLT-100
- Dependent on RLT-002 MARL passing
- Expectation being match will be configured to have 2 pre-trained RL agents of the same model

Expected Outcomes

- Tracked historical wins and losses on average stays similar by some tolerance
- Rendered match of either agent demonstrates observably different behaviour when compared to their previous behaviour before self-training
 - Can be further validated by rendering matches against the version of the agent that didn't go through self-training vs the self-trained agent one, so expected output would be the self-trained agent winning more matches on average
 - Either agents shouldn't be allowed to train off visualized match results in this case

RLT-20X Multi Agent Reinforcement Learning(MARL)

RLT-200 MARL

Input

- Refer to RLT-000 input
- Exception being match is configured to run RL agent Vs RL agent
- Both opposing agents are the identical model
- Both agents haven't been trained yet

Outcomes

- Refer to RLT-000 outcomes.
- Generally, RL interfaces, game states, and agent parameters should differ by some tolerance to indicate functionality.

Section 2.2 Reinforcement Learning Configuration (RLC)

RLC - 00X

RLC - 000 Custom Agent Implementation

Input

- Refer to RLT-000 input for default testing configurations
- Exception being that, from GUI a different option for RL agent model can be chosen and is chosen to train against a scripted opponent

Outcomes

- Refer to RLT-000 outcomes
- Exception being that we observe that the interface is correctly calling the alternative algorithm associated with the chosen model.
 - Still measure tolerances to see if algorithm is working in a basic way as expected

RLC - 001 Match Configuration

Input

- A set of different configurations from the configuration file is selected
 - Base default settings specified in RLT-000 input
 - Change the ring(add static obstacles for example)

- Change physics-related properties (such as gravity)

Outcomes

- Using a debugger or observing debug logging, related physics properties and ring alterations match the settings set in the configuration files and for match alterations that add physics objects. They can be verified via simple observations and other properties that can be debug values such as “debug.assert(NumStaticBodies == config.NumObstacles)”

Section 2.3 Physics Engine Tests (PET)

PET-00X

PET-000 Agent Visualization

Input

- Base default settings specified in RLT-000 input
- Exception being that RL agent is configured to have an alternative body as set by some modular body customization system

Outcomes

- The modified agent visually has the correct modular body set-up and displays expected physical behavior for the body it has